Distributed Multi-Model Analytics for E-commerce Data

Technical Report - MongoDB, HBase and Apache Spark

Your Full Name | Your Student ID | Big Data Analytics - AUCA | 11 November 2026

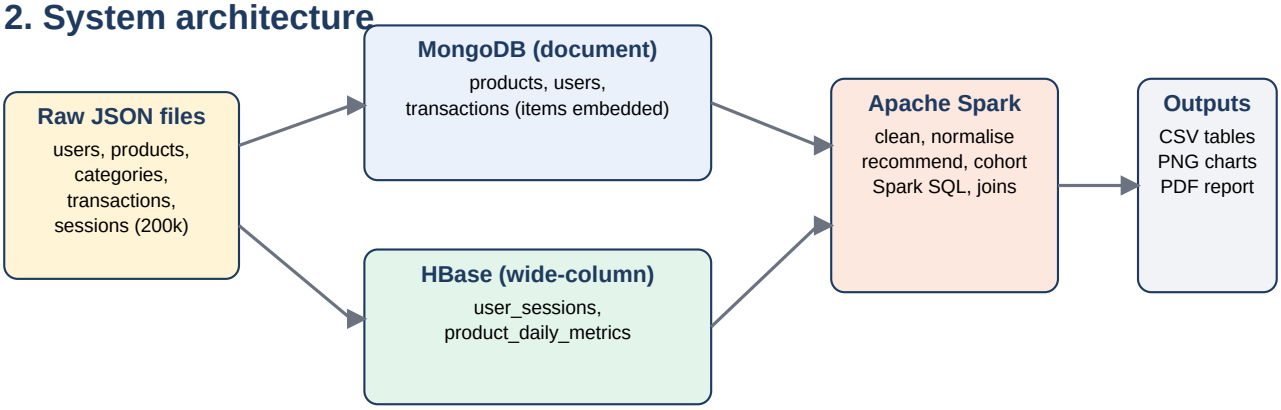
Code repository: https://github.com/YOUR-USERNAME/YOUR-REPO

1. Introduction and objectives

This project builds a multi-model analytics system for a synthetic e-commerce platform. Each technology is used for the workload it handles best: **MongoDB** stores transactional and catalogue data as documents, **HBase** stores the high-volume browsing time series, and **Apache Spark** cleans the data, computes analytics and joins the two stores. The questions answered are: which products and categories earn the revenue, which customers matter most, which products are bought or viewed together, and where shoppers leave the purchase funnel.

Entity	Records	Notes
Users	10,000	profile, geography, registration and last-active dates (no age field in the generator)
Products / categories	5,000 / 25	price history embedded; products carry a category but no subcategory
Sessions	200,000	page views, cart contents, device, referrer; about 90 days of activity
Transactions	50,000	embedded line items; 24,906 (49.8%) have no linked session

2. System architecture



The JSON files are loaded into two stores according to their access pattern. Transactions, users and products go to MongoDB because they are nested documents queried with flexible aggregations. Sessions go to HBase because they are an append-only time series that is read by user and by time range. Spark reads the raw files for large batch jobs and reads from both databases (through Python extraction) for the integrated query. Spark writes small result tables as CSV; Python then draws the charts for this report. MongoDB and HBase run in Docker containers started by one `docker compose up` command, so the whole project is reproducible.

3. Data modelling (Part 1)

3.1 MongoDB document model

Collection	Design decision	Reason
transactions	Line items **embedded** as an array; `category_id` copied into every item; timestamps stored as real dates	A transaction is always read with its items (one read, no join). Copying the category makes revenue-by-category a single pipeline with no lookup.
products	Price history embedded; category name copied into the product	Always needed together with the product; avoids a join in every product query.
users	Profile plus geography embedded, and a **pre-computed purchase_summary** (orders, total spent, first and last purchase)	Segmentation reads one small document per user instead of scanning 50,000 transactions.
categories	Separate reference collection with subcategories embedded	Small and rarely changing; referenced by `$lookup` only for display names.

Trade-off. Embedding and copying make reads fast but duplicate data: if a category were renamed it would have to be updated in several places. This is acceptable for an analytics dataset that is loaded in bulk. Indexes were created on *transactions.timestamp*, *transactions.user_id*, *transactions.items.product_id*, *products.category_id* and the user order count, matching the query patterns. Sessions were deliberately **not** stored in MongoDB: at 200,000 documents with long page-view arrays they are a write-heavy time series.

Aggregation pipelines. (1) *Revenue by month and category*: `$unwind` on items, `$group` by category and month (`$dateToString`), `$lookup` for the category name. (2) *User segmentation by purchase frequency*: `$bucket` on the order count with average and total spend per band. (3) *Top-selling products*: unwind, group by product, sort, lookup. Age-group segmentation was not possible because the generator creates no age, so purchase frequency was used instead.

3.2 HBase wide-column model

Table	Row key	Column families	Why
user_sessions	`user_id#reverse_timestamp` (Long.MAX minus start time in ms, zero-padded to 19 digits)	**info**: session metadata; **pv**: one column per page view (pv:000, pv:001, ...); **cart**: one column per product	All sessions of a user are adjacent, so one prefix scan returns them; HBase sorts keys ascending, so the **newest session comes first**. Sessions have different numbers of events, so rows are sparse and only existing cells are stored.
product_daily_metrics	`product_id#yyyymmdd`	**m**: views, cart_adds, purchases	A product's history over a date range is one contiguous scan; counts are pre-aggregated per day.

Separate column families keep data that is read together physically together: a query that only needs device and referrer reads the *info* family and never touches the page-view cells. Implemented queries: latest N sessions of a user (prefix scan with a limit), sessions of a user inside a time range (start/stop rows computed from the reversed timestamps), the full journey of the latest session, and view-to-cart and cart-to-purchase rates of a product over a date range. The HBase shell commands for the schema and queries are in the repository. The loaded subset holds 100,000 sessions (the first generated file) and 236,715 product-day rows.

3.3 Why some data is in MongoDB and some in HBase

Criterion	MongoDB	HBase
Data shape	Nested business documents (transactions with items, users, catalogue)	Event streams: sessions and page views, sparse and variable length
Main access pattern	Filters, grouping and joins across many fields (aggregation framework)	Lookup by key and range scans over time (user and day)
Growth	Moderate (50,000 transactions)	Very high and append-only (200,000 sessions, millions of events)
Weak point	Large write-heavy time series	No flexible ad-hoc queries; the row key must match the query

4. Data processing with Apache Spark (Part 2)

All Spark jobs share one cleaning module (*common.py*), so every analysis starts from the same cleaned DataFrames. Explicit schemas are used instead of schema inference; the files are single JSON arrays, so they are read with the multi-line option, and the cart contents are modelled as a *map* type because every cart has different product keys.

4.1 Cleaning and normalisation (batch job 1)

- Removed duplicates and null keys, cast text dates to timestamps, trimmed and lower-cased status, payment and referrer fields, filled missing geography and device values with explicit "unknown" labels.
- Added quality flags and a data-quality report. Findings: **24,906 transactions without a session** (flag *is_orphan*); **15,515 sessions labelled "converted" but with no transaction** (38% of all converted labels); **163 products active with zero stock**; no duplicates, bad durations, amount mismatches or inconsistent dates.
- Cause of the false labels (confirmed by a per-file check): the first 100,000 sessions are fully consistent (20,239 converted, all with a transaction); in the second file only 4,855 of 20,370 converted sessions have a transaction, with the first mismatch around session 124,000. The generator stops creating transactions at its 50,000 limit but keeps labelling sessions as converted. The cleaning job therefore derives **verified_conversion** by joining sessions with transactions and uses it instead of the label.

4.2 Recommendation indicators (batch job 2)

For two items A and B in a basket (a transaction or a session) the job computes **support** (share of baskets with both), **confidence** A to B (baskets with both divided by baskets with A, i.e. "users who bought A also bought B") and **lift** (support divided by the product of the individual supports; above 1 means they appear together more than chance). It is implemented as a self-join inside each basket with the condition A less than B, followed by a join to the item counts. It was run for products bought together (50,000 baskets), products viewed together (197,422 sessions) and category affinity.

4.3 Cohort analysis (batch job 3) and Spark SQL

Users are grouped into cohorts by registration month. For each cohort and each month offset (whole months between registration and purchase) the job computes revenue, active buyers, spend per cohort user and buyer rate. Spark SQL queries on views named after the system they represent (*mongo_users*, *mongo_transactions*, *hbase_sessions*) cover: top three categories per month (window function *ROW_NUMBER*); conversion by device and referrer; customer value quartiles joining users, transactions and sessions (*NTILE*, CTEs); and a page-level funnel using the higher-order function *exists* on the page-view array.

4.4 Optimisation choices

- Small dimension tables (categories, products, transaction session ids) are **broadcast**; the physical plan of a test query shows *BroadcastHashJoin*.
- Sessions are cached once (memory and disk) because they are reused many times; shuffle partitions are reduced from the default 200 to 16 for a single machine.
- Rare pairs are filtered before the second join to reduce shuffled data; analysis scripts skip the data-quality counting jobs; small results are saved with pandas to avoid Hadoop write problems on Windows.

5. Integrated analytics workflow (Part 3)

Item	Description
Business question	Where do shoppers leave between browsing and paying, and do device, week or product category change the conversion, using only **verified** purchases?
Sources	**HBase** user_sessions (page views, cart, device, referrer); **MongoDB** transactions (what was really bought) and products (category); **Spark** for the join and aggregation.
Steps	1) scan HBase for the three column families needed; 2) query MongoDB for only the needed fields; 3) load both into Spark DataFrames with explicit schemas; 4) left join sessions with transactions on session_id; 5) aggregate the funnel overall and by device, week and category (view, cart, purchase per category).
Why these technologies	HBase serves the large session time series efficiently; MongoDB is the source of truth for purchases; Spark performs the cross-system join that neither database can do alone.
Performance notes	The HBase scan reads only the needed families in batches of 500 rows; the MongoDB query projects only the needed fields; the small transaction table is broadcast. Extraction through one Python process is the bottleneck; at scale the HBase-Spark and MongoDB Spark connectors would load the data in parallel.

On the 100,000 HBase sessions the integrated funnel reached 20.2% verified purchases, with 20.2-20.3% on every device and 19.7-21.2% in every full week; cart-to-purchase by category ranged from 29.4% to 32.3%. The sample is the first generated file, which contains no false labels, so its rate is higher than the full-dataset 12.5% computed by Spark; this gap is explained in Section 4.1 and in the limitations.

6. Results, visualisations and business insights (Part 4)

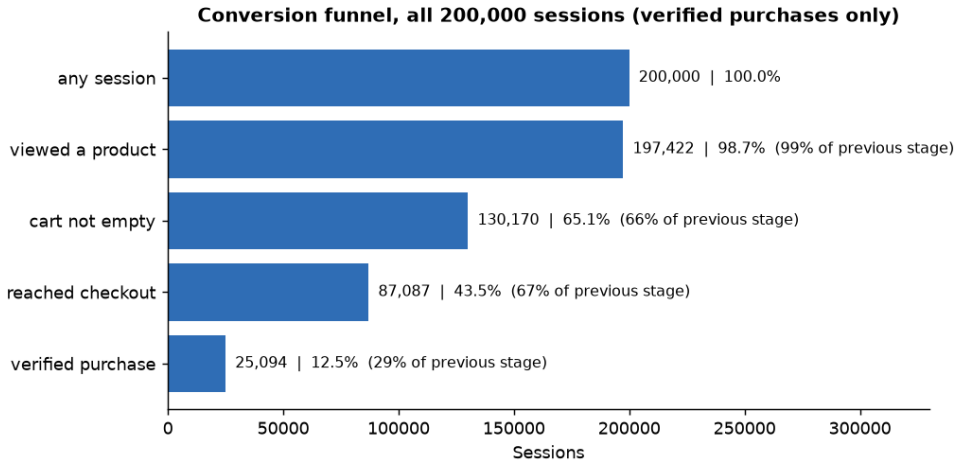


Figure 1. Conversion funnel for all 200,000 sessions, counting only purchases verified by a MongoDB transaction.

Funnel. 98.7% of sessions view a product, 65.1% fill a cart (66% of viewers), 43.5% reach checkout (67% of carts) and only **12.5%** buy (28.8% of checkout sessions). The raw label would suggest 20.3%, an overstatement of 38%. The weakest step is checkout to purchase: about 62,000 sessions reached checkout and did not buy.

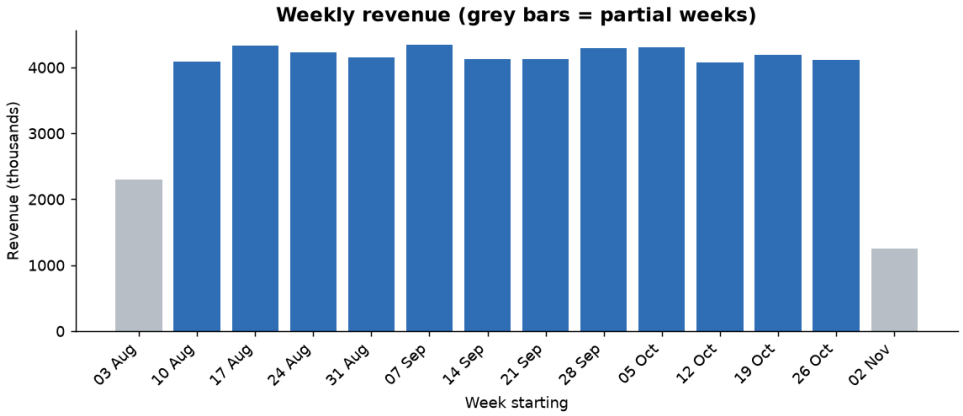


Figure 2. Weekly revenue from the MongoDB transactions (grey bars are partial first and last weeks: the data starts in early August and ends on 3 November).

Sales over time. Weekly revenue is steady with no strong trend, as expected for randomly generated data; the partial weeks must not be compared with full ones. By category, *Kelly-Santiago* is the top earner in August, September and October (about 756k, 883k and 937k), but the next categories are close behind, so the ranking is not decisive.

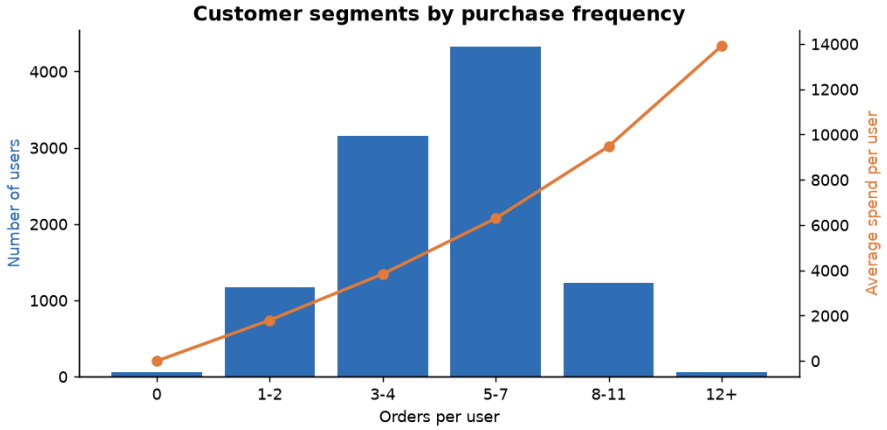


Figure 3. Customers by number of orders (bars) and average spend per user (line), computed from the cleaned transactions.

Orders per user	Users	Average spend	Share of users	Share of revenue
0	58	0	0.6%	0%
1-2	1,176	1,804	11.8%	3.9%
3-4	3,155	3,840	31.6%	22.5%
5-7	4,326	6,298	43.3%	50.5%
8-11	1,226	9,481	12.3%	21.6%
12+	59	13,916	0.6%	1.5%

Segmentation. Spend per user rises from about 1.8k to 13.9k as orders increase, while the average order value stays near 1,080 (total revenue of about 53.9 million over 50,000 orders). Revenue is therefore driven by **how often** customers buy, not by basket size. The 5-7 order segment is 43% of users and 50% of revenue. The value-quartile query agrees: the top quartile orders 7.5 times and spends 9,253 per user against 2.7 orders and 2,034 in the bottom quartile, while sessions per user (19-21) and session length (about 1,810 seconds) barely differ.

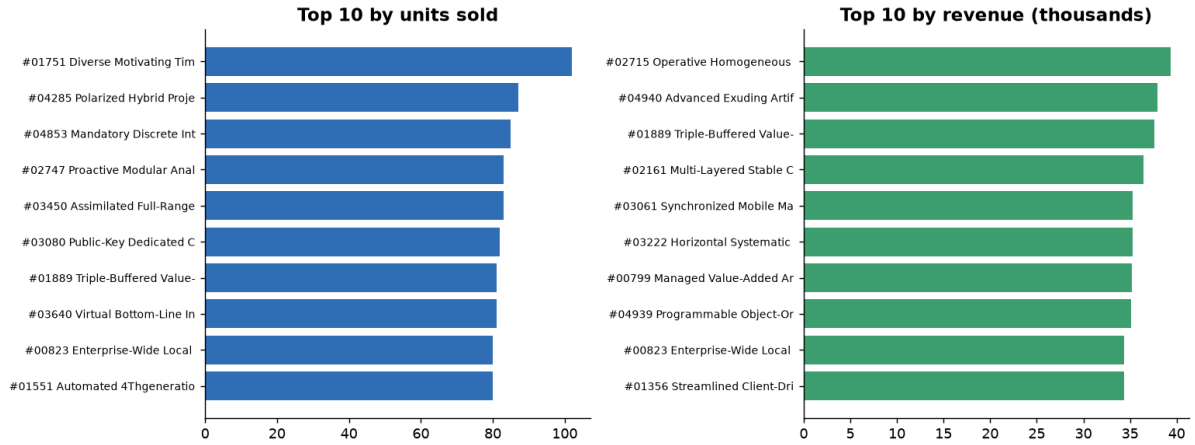


Figure 4. Top 10 products by units sold (left) and by revenue (right).

Products. Ranking by units and by revenue gives different lists: the best-seller by units (prod_01751, 102 units) earned 33,779, whereas prod_04285 sold 87 units but earned only 1,786 because it is cheap. Category affinity lift is 0.80-0.85 and product-pair lifts are inflated by counts of 2-5 pairs, so the data contains **no real affinity**; the method is valid but cannot recommend from random data.

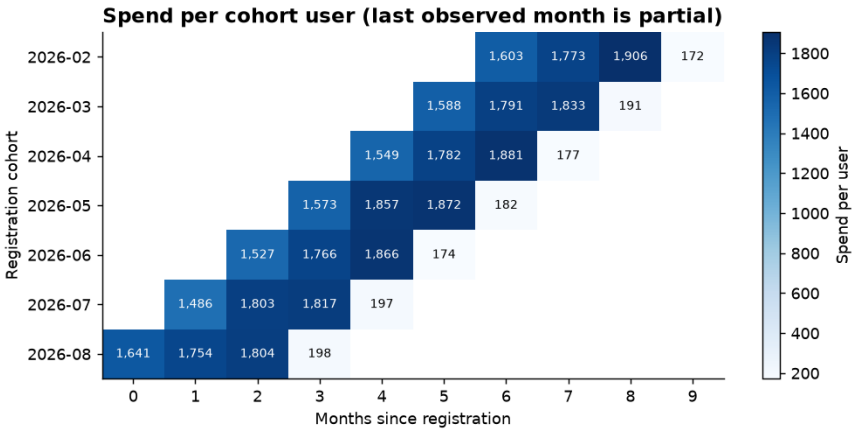


Figure 5. Spend per cohort user by registration month (rows) and months since registration (columns); the last observed month of each row is partial.

Cohorts. Cohorts are Feb-Aug 2026 (1,241 to 1,745 users each; August only 280 because registrations end in August). In their first full months 76-85% of each cohort buys and spend per user is about 1,490-1,910; the drop to 13-18% in the last column is an artefact of the incomplete final month, not churn.

Actionable insights

Finding	Recommended action
Only 28.8% of checkout sessions end in a purchase (about 62,000 lost)	Simplify checkout, add payment options, and send recovery emails to abandoned checkouts; measure this step first.
Frequency drives revenue; 5-7 order customers give half of revenue	Loyalty or reminder campaigns that move 3-4 order customers to 5-7 orders; reactivate the 1,176 customers with only 1-2 orders.
Units and revenue rankings differ	Rank and promote products by revenue or margin, not units; review cheap high-volume items.
The converted label overstates conversion by 38%; half of the orders have no session	Fix event tracking and order-to-session attribution before using the label for decisions.
Device, referrer, week and category conversions differ by only 1-3 points	Do not shift budget by device or channel on this evidence; test with controlled experiments.
163 active products have no stock	Deactivate or restock them so shoppers do not reach unavailable items.

7. Methodology of the key analyses

- **Verified conversion:** a session is converted only if a transaction with its session_id exists. Funnel stages are counted per session from the page-view and cart data.
- **Revenue:** gross item revenue (before the transaction-level discount) for category and product results; transaction totals (after discount) for user spend and segments.
- **Segments:** users are bucketed by order count into 0, 1-2, 3-4, 5-7, 8-11 and 12+; the bands were chosen to give groups large enough to compare (total users 10,000).
- **Lift:** support(A and B) divided by support(A) times support(B); values are only meaningful when pair counts are large, so category-level lift is the reliable view.
- **Cohort offset:** whole months between the registration month and the purchase month; partial final months are flagged, not removed.

8. Scalability considerations

- **Spark:** the job code is cluster-ready (DataFrames, no driver-side loops). The generator's JSON files are single arrays, which Spark cannot split, so each file is read by one task; at scale the data should first be converted to JSON Lines or Parquet and partitioned by date. The full 2,000,000-session size would need a cluster or Parquet input.
- **MongoDB:** use a replica set for availability and shard transactions by a hashed user_id (or by time) to spread writes; keep the embedded design so a transaction stays on one shard.
- **HBase:** regions split automatically as data grows. Sequential user ids can create hotspots, so a hash or salt prefix on the row key would be added at very large scale while keeping the reverse timestamp for recency.
- **Integration:** replace the Python extraction with the HBase-Spark and MongoDB Spark connectors so that reading is parallel and filters are pushed down to the databases.

9. Technology selection justification

Analytical task	Technology	Justification
Revenue, product popularity, user segments	MongoDB aggregation	Nested transactions, grouping, bucketing and lookups in one pipeline over documents
Latest sessions of a user, time-range scans, product-day metrics	HBase	Key-ordered storage: reverse-timestamp row keys give recent-first reads without sorting
Cleaning, recommendations, cohorts, SQL over all sessions	Spark	Distributed joins and aggregations over data larger than one process, with SQL and DataFrame APIs
Funnel across behaviour and purchases	HBase + MongoDB + Spark	Behaviour and transactions live in different systems; only Spark can join them

10. Limitations and future work

- **Synthetic, random data:** differences between categories, devices and weeks are noise, and lift values show no real affinity. The analyses demonstrate the methods, not real market findings.
- **Generator inconsistencies:** half the transactions are not linked to a session, 15,515 converted labels are false, and there is no user age or product subcategory; these were flagged rather than repaired.

- **Short history:** only about 90 days, so cohorts are short and the last month is partial. No statistical tests were run; with about 4,000 cart sessions per category a 2-3 point difference can be chance.
- **HBase subset:** a single-laptop Docker HBase crashed under the full load, so 100,000 sessions (the first generated file) were loaded. Its funnel (20.2%) is not representative of the full data (12.5%). A systematic sample over both files was added to the loader as a fix; the full-dataset funnel was computed in Spark.
- **Future work:** connector-based integration, Parquet storage, a collaborative-filtering recommender (ALS) on real data, customer lifetime value modelling, a streaming simulation with Spark Structured Streaming, and significance testing of conversion differences.